Abid	TP N°6	Enseignante : Arij

TP Django : Gestion des Auteurs et des Articles

Objectif :

- Comprendre la structure d'un projet Django (models, views, urls, templates)
- Créer et manipuler des **modèles liés (One-to-Many)**
- Implémenter un **CRUD complet** (Create, Read, Update, Delete)
- Utiliser les **ModelForm** pour simplifier la gestion des formulaires
- Comprendre le rôle de ForeignKey, related_name, get_object_or_404

Contexte du TP :

Vous devez développer une application Django permettant de gérer :

- des **auteurs** (nom, prénom, email)
- des **articles** rédigés par ces auteurs

Chaque article est obligatoirement écrit par **un seul auteur**, tandis qu'un auteur peut écrire **plusieurs articles**.

L'application devra permettre :

- la gestion complète des auteurs
- la gestion complète des articles
- l'affichage des relations entre auteurs et articles

1. Partie 1 – Création des modèles

a. Créer un modèle Author :

Contraintes :

- nom : texte (100 caractères max)
- prénom : texte (100 caractères max)
- email : unique

```
class Author(models.Model):
    nom = models.CharField(max_length=100)
    prenom = models.CharField(max_length=100)
    email = models.EmailField(unique=True)

    def __str__(self):
        return f"{self.prenom} {self.nom}"
```

b. Créer un modèle Article lié à l'auteur :

Contraintes :

- **Un article possède un auteur**
- Un titre
- Un contenu
- Une date de publication automatique

```
class Article(models.Model):
    auteur = models.ForeignKey(
        Author,
        on_delete=models.CASCADE,
        related_name='articles'
    )
    titre = models.CharField(max_length=100)
    contenu = models.TextField()
    date_pub = models.DateTimeField(default=timezone.now)

    def __str__(self):
        return self.titre
```

related_name='articles' permet d'accéder aux articles d'un auteur via author.articles.all().

- CharField : chaîne de caractères, ici max 200 caractères.
- TextField : texte long.
- auto_now_add=True : la date est ajoutée automatiquement lors de la création.

- __str__ : permet d'afficher le titre dans l'admin Django ou dans les listes.

Appliquer les migrations

Pour créer la table Article dans la base de données :

python manage.py makemigrations

python manage.py migrate

▶ Le code complet du modèle doit être identique à celui présenté dans cette capture.

models.py

FirstApp > models.py

```python
from django.db import models
from django.utils import timezone


class Author(models.Model):
    nom = models.CharField(max_length=100)
    prenom = models.CharField(max_length=100)
    email = models.EmailField(unique=True)

    def __str__(self):
        return f"{self.prenom} {self.nom}"

class Article(models.Model):
    auteur = models.ForeignKey(
        Author,
        on_delete=models.CASCADE,
        related_name='articles'
    )
    titre = models.CharField(max_length=100)
    contenu = models.TextField()
    date_pub = models.DateTimeField(default=timezone.now)

    def __str__(self):
        return self.titre

```

2. Partie 2 – Création des formulaires

a. Créer un formulaire Django pour gérer **les articles.**

b. Créer un formulaire pour gérer **les auteurs.**

forms.py

FirstApp > forms.py

```python
from django import forms
from .models import Article, Author

class ArticleForm(forms.ModelForm):
    class Meta:
        model = Article
    #   fields = ['titre', 'contenu']
        fields = '__all__'




class AuthorForm(forms.ModelForm):
    class Meta:
        model = Author
        fields = ['nom', 'prenom', 'email']
```

1. **Partie 3 – Routage (urls.py)**

   Définir les routes permettant :

   - ✓ lister
   - ✓ créer
   - ✓ modifier
   - ✓ supprimer
   - ✓ afficher les détails

     pour les articles et les auteurs.

urls.py

FirstApp > urls.py

```python
from django.urls import path
from . import  views

urlpatterns = [
#articles
    path('liste', views.liste_articles, name='liste_articles'),
    path('create', views.create_articles, name='create_articles'),
    path('update/<int:pk>/', views.update_articles, name='update_articles'),
    path('delete/<int:pk>/', views.delete_articles, name='delete_articles'),
    path('article/<int:id>/', views.details_article, name='details_article'),

#authors
     path('authors/', views.list_authors, name='list_authors'),
     path('authors/create/', views.create_author, name='create_author'),
     path('authors/update/<int:pk>/', views.update_author, name='update_author'),
     path('authors/delete/<int:pk>/', views.delete_author, name='delete_author'),
     path('authors/details/<int:id>/', views.details_author, name='details_author'),
]

```

4. Partie 4 – Vues Articles (CRUD)

a. Afficher la liste des articles avec leurs auteurs :

```
#********************Article ************************
def liste_articles(request):
        articles = Article.objects.select_related('auteur')
        #articles = Article.objects.all()
        return render(request, 'liste_articles.html', {'articles': articles})
```

⇨ **select_related** améliore les performances en évitant des requêtes inutiles.

b. Création d'un article :

```
def create_articles(request):
    if request.method == 'POST':
        form = ArticleForm(request.POST)

        if form.is_valid():
            form.save()
            messages.success(request, "✅ L'article a été créé avec succès !")
            return redirect('liste_articles')
        else:
            messages.error(request, "Erreur ❌ : veuillez vérifier le formulaire")
    else:
        form = ArticleForm()

    return render(request, 'create_articles.html', {'form': form})
```

c. Modification d'un article :

```
# updpate view
def update_articles(request, pk):
    article = get_object_or_404(Article, id=pk)

    if request.method == 'POST':
        form = ArticleForm(request.POST, instance=article)

        if form.is_valid():
            form.save()
            messages.success(request, "✏️ L'article a été mis à jour avec succès !")
            return redirect('liste_articles')
        else:
            messages.error(request, "Erreur ❌ : veuillez vérifier le formulaire")
    else:
        form = ArticleForm(instance=article)

    return render(request, 'update_articles.html', {'form': form})
```

d. **Suppression d'un article :**

```
# delete view
def delete_articles(request, pk):
    article = Article.objects.get(id=pk)

    if request.method == "POST":
        article.delete()
        return redirect('liste_articles')
```

e. **Détails d'un article :**

```
def details_article(request, id):
    article = get_object_or_404(Article, id=id)
    return render(request, 'details_article.html', {'article': article})
```

5. PARTIE 5 – Gestion des auteurs :

a. Liste des auteurs

```
#************************author ************************
def list_authors(request):
    authors = Author.objects.all()
    return render(request, 'authors/list.html', {'authors': authors})
```

b. Détails d'un auteur :

```
# Détails d'un auteur
def details_author(request, id):
    author = get_object_or_404(Author, id=id)
    return render(request, 'authors/details_author.html', {'author': author})
```

c. **Création d'un auteur :**

```
def create_author(request):
    if request.method == 'POST':
        form = AuthorForm(request.POST)
        if form.is_valid():
            form.save()
            messages.success(request, "✅ L'author a été créé avec succès !")
            return redirect('list_authors')
        else:
            messages.error(request, "Erreur ❌ : veuillez vérifier le formulaire")
    else:
        form = AuthorForm()

    return render(request, 'authors/create.html', {'form': form})
```

d. **Modification d'un auteur :**

```
# Modification d'un auteur
def update_author(request, pk):
    author = get_object_or_404(Author, id=pk)
    if request.method == 'POST':
        form = AuthorForm(request.POST, instance=author)
        if form.is_valid():
            form.save()
            messages.success(request, "Auteur modifié avec succès !")
            return redirect('list_authors')
    else:
        form = AuthorForm(instance=author)
    return render(request, 'authors/update_author.html', {'form': form})
```

e. **Suppression d'un auteur :**

```
def delete_author(request, pk):
    author = get_object_or_404(Author, id=pk)

    if request.method == "POST":
        author.delete()
        messages.success(request, "Auteur supprimé avec succès !")
    return redirect('list_authors')
```